EduVerify: Feature and USP Analysis

Repository assessment · 30 October 2026 · source snapshot `769ce2e`

Download the PDF · Project README · Website

1. Executive assessment

EduVerify is a synthetic student-data governance prototype for the SIH26263 problem context. Its strongest proposition is the combination of explainable ingestion, privacy-aware duplicate investigation, approved-intake enforcement and permissioned evidence release. The public product is named EduVerify; WEAVE remains the internal implementation name.

The repository is more mature than a frontend-only concept: it contains FastAPI services, PostgreSQL models, five Alembic migrations, backend policy enforcement and real Ed25519 signing. It is less mature than an integrated production platform: the React client still assumes fixture identities, calls legacy routes and keeps some student actions in browser memory. The right description is **a substantial backend foundation with a polished synthetic portal and incomplete integration**.

Recommended one-line pitch: EduVerify helps institutions turn student submissions into explainable, capacity-aware records, investigate possible duplicates and release consent-scoped enrolment evidence.

What makes the proposition compelling: it addresses the transitions between data collection, review and evidence release. A dashboard alone does not prevent over-capacity commits; a signature alone does not prove a source record is correct; an identity match alone does not justify a fraud finding. EduVerify contains mechanisms addressing each of those boundaries, although the full chain still needs integration and assurance.

2. Scope and evidence standard

This assessment inspected application routing, authentication adapters, fixture state, Express endpoints, FastAPI routes/services/policies, database models/migrations, tests, configuration and architecture documents. Source links below are relative to this report. Repository: Sarthaksulkhlan/EduVerify.

Evidence labels used throughout:

- **Implemented backend:** the behavior exists in FastAPI code; this alone is not proof of deployment or complete integration.

- **Demonstration:** the UI or Express fixture implements an illustrative workflow using synthetic or local state.
- **Integration gap:** both concepts may exist, but the UI does not call the implemented backend contract correctly.
- **Planned:** documented intent without a complete runnable implementation found in the inspected snapshot.
- **Inference:** an architectural implication or risk derived from code; it requires a targeted test before being treated as a reproduced defect.

Observed checks: the public homepage returned HTTP 200; `/health` and `/api/v1/health` returned synthetic health JSON; unauthenticated `/api/v1/auth/me` returned 401. A browser captured the homepage. These checks do not establish database availability, credential issuance, account provisioning or authenticated portal correctness. No production credentials or personal data were used.

The backend suite completed with **39 passed and 4 skipped**. The skipped tests require a disposable PostgreSQL database. Node.js was not available on PATH in the review environment, so frontend build, type checking and engine tests were not run. No latency, scalability, accessibility or market-comparison benchmark was performed.

3. Users, problems and value

User	Current problem addressed	Relevant capability	Boundary
Institution operator	Repeated uploads and unclear record errors	CSV validation, row findings, idempotent ingestion	Needs correct programme/year reference data
Institution administrator	Enrolments can exceed approved pools	Transactional capacity recheck at commit	Database concurrency behavior still needs execution evidence
Regulator administrator	Cross-institution overlaps need adjudication	Masked duplicate cases and decision history	A candidate is not proof of misconduct
Student	Limited visibility into records and sharing	Personal portal, discrepancy and consent experiences	Several UI actions are browser-local
External verifier	Full record sharing exposes unnecessary information	Purpose/claim grants plus consent and signed credentials	No official identity-service connectivity
Auditor	Actions are difficult to reconstruct	Actor-linked, hash-linked backend events	Timeline UI and backend endpoint coverage are incomplete

Institutional reference data and accountable onboarding are prerequisites. A platform can validate consistency against its configured rules while the underlying capacity approval, institution membership or source record remains wrong. Official assurance therefore requires trustworthy upstream data and authorization beyond this

prototype.

4. Detailed capability inventory

4.1 CSV ingestion and explainable validation

Implemented backend. The parser accepts CSV content through a JSON upload, requires seven headers and recognizes eight optional admission/category fields. It rejects unknown/duplicate headers and Aadhaar-like columns. Limits are 5 MiB, 10,000 data rows and 16,384 characters per field. File-structure failures are separate from row-value findings. Spreadsheet-dangerous output values are neutralized in error exports.

Required headers: `student_ref`, `apaar_id`, `full_name`, `dob`, `program_code`, `enrollment_year`, `consent_granted`. Missing APAAR can yield a provisional identity and warning rather than an automatic rejection. This is inclusive data handling, not proof that an unverified identity is valid.

Upload fingerprints are keyed, institution-scoped and based on canonical content. Renaming a file or changing column order does not alone create a new logical batch. Fingerprints are key-version-sensitive; changing keys without a migration can alter repeat-upload behavior. Canonicalization should not be advertised as deduplicating every semantically equivalent CSV, including arbitrary row permutations, without testing that case.

Value: fewer accidental repeated imports and actionable correction feedback. **Boundary:** UI wording mentioning Excel is not evidence of an `.xlsx` parser. Current backend ingestion is CSV.

Evidence: CSV parser, ingestion service, ingestion routes, CSV tests.

4.2 Commit-time intake enforcement

Implemented backend; database execution not verified in this review. Commit accepts eligible READY/WARNING rows, locks relevant records, rechecks programme/year total and category pools, and writes enrolments transactionally. Review-required/rejected rows are excluded. A capacity conflict blocks the attempted commit set rather than partially admitting it. A repeated successful commit returns zero newly committed rows.

Value: capacity is treated as a write-time invariant rather than only a chart. This matters when available seats change between upload preview and final commit.

Tradeoff: correctness-oriented locking may serialize uploads at institution/programme boundaries. Benchmark contention before making throughput claims. Category configuration must be explicit; nationality flags must not silently choose a seat pool. The displayed dashboard and enforcement service should share the same definitions.

Evidence: commit service, PostgreSQL ingestion tests, academic models.

4.3 Privacy-aware duplicate investigation

Implemented backend; UI integration gap. Exact APAAR linkage uses keyed identity references. Institution-local student references are compared within the same programme/year. Secondary matching groups candidates by domain-separated DOB HMAC and compares normalized names using `SequenceMatcher`, with a default threshold of 0.88. Confidence bands are deterministic heuristics, not calibrated probabilities.

Case responses contain masked fields, source institution references, reason codes and explanations. Raw names and dates of birth are processed inside the service, not exposed as full case details. Missing APAAR is not a matching signal. Gender and nationality do not drive matching.

Only regulator administrators record outcomes: `CONFIRMED_DUPLICATE`, `DISTINCT_STUDENTS`, `INSUFFICIENT_EVIDENCE` or `ESCALATED`. Rescans preserve human decisions. The UI currently calls `/conflicts`, while FastAPI exposes `/students/duplicates`; the current interface cannot be counted as full integration with this service.

Tradeoffs: common names and incorrect DOB values can cause false positives or misses. Pairwise work within large blocks can grow quadratically. The scan loads candidate records in memory; no national-scale performance claim is justified. Decision concurrency and escalation policy need explicit tests.

Evidence: matching and review service, duplicate routes, matching tests, client adapter.

4.4 Authentication and authority

Implemented backend; significant frontend gap. FastAPI uses salted scrypt passwords, opaque 15-minute access tokens stored as digests, and rotated refresh tokens in HttpOnly/SameSite=Strict cookies with an eight-hour lifetime. Accounts are provisioned through a local CLI. Roles and institution scope come from the authenticated database principal rather than browser-submitted role fields. Login throttling is process-local.

The frontend login adapter still replaces backend user information with synthetic accounts. The student branch creates a synthetic token after the backend login; the admin branch can choose a fixture regulator identity. FastAPI still checks its own token/principal, so this finding is **not proof of backend privilege escalation**. It is an integration and misleading-identity problem that can cause authorization failures and incorrect UI context.

Acceptance requirement: every successful live login must retain the exact server-issued token and user identity; reload/refresh must retain the same authorized context; student A must never see student B's fixture fallback.

Evidence: backend auth, policy layer, frontend login, login screen.

4.5 Consent and signed record checks

Implemented backend; legacy UI gap. A regulator administrator grants a verifier purpose/claim permission. A linked student grants consent for an enrolment, verifier, purpose and claim subset. Issuance checks both authorization and current consent. Consent lasts 30 days in the service; generated credentials last five minutes, bind to an audience and contain a random credential identifier.

FastAPI signs a custom `WEAVE-V1` credential using real Ed25519. Verification checks encoding/signature, issuer, audience, expiry, stored credential status and the requesting verifier. The credential explicitly identifies its source as `WEAVE_RECORD_CHECK` and external verification as `NOT_PERFORMED`. It should not be described as a standards-certified W3C credential or an official APAAR result.

Consent withdrawal revokes outstanding credentials for the same enrolment/verifier. This is broader than a single purpose because the stored credential does not carry a purpose link. Verifier-grant revocation marks the grant revoked; the current credential-check path does not recheck grants. **Inference:** an existing credential may remain accepted until expiry after grant revocation. Test and define the intended semantics.

Two important inconsistencies need reconciliation. Policy/CLI claims include `consent_status`, while the signing service accepts `award_status` instead. Their common working vocabulary is `enrolment_status`, `program_code`, `academic_year`. Also, `award_status` is derived from an accepted enrolment, not a separate academic award record. It must not be presented as degree or award verification.

The Express attestation demo is separate: its deterministic hash is labelled with an `ed25519:` prefix but is not real Ed25519 signing. Public `/verify/:token` and legacy `/attestations` do not map to the authenticated FastAPI record-check API.

Evidence: verification service, privacy routes, authorization, credential model, privacy tests, demo engine.

4.6 Auditability and privacy

Implemented backend foundation; incomplete operational assurance. Audit services link actions to actors, resources and request IDs. The privacy service uses a metadata allowlist and SHA-256 hash-linked events. Tests cover tampering. Ingestion also has its own audit-writing path; consistent serialization, verification and coverage across both paths deserve review.

Hash linkage is tamper-evident only relative to a trusted history or anchor. It does not make a database immutable: an actor able to rewrite a whole chain could recompute hashes. Concurrent first writes and tail locking need stress tests. Duplicate review stores decision history but does not call the shared audit writer in the inspected service. No legacy `/audit-events` timeline endpoint exists in FastAPI.

HMAC linkage reduces exposure of raw APAAR values in matching references. It is not anonymization of the whole database. Import rows retain personal fields such as names and DOB. Retention, deletion, backups, access review and operational key rotation remain necessary work. No legal-compliance certification is inferred from these controls.

Evidence: audit service, ingestion model, ingestion audit writer, engineering requirements.

4.7 Portals, reporting and assistant

Demonstration with partial API usage. Student pages cover dashboard, profile, academic records, APAAR, verification, discrepancies, consent, history and help. Administration pages cover institutions, students, uploads, validation, duplicates, intake, verification, reports and audit logs.

`portalRuntimeState` holds seeded academic records, requests, discrepancies, consent preferences and local events. Some workflows read legacy APIs before applying local mutations. A successful click or toast is therefore not proof of durable persistence. Refresh/reload tests are essential.

The assistant has deterministic advisory responses and optional Gemini calls in Express. There is no matching FastAPI assistant route. Model labels in the UI do not prove provider availability. It is not an AI fraud detector: deterministic validation and authorization must remain outside the LLM. Three.js components add presentation value but the old topology route is not registered in the current router.

Evidence: routes, client state, student pages, admin pages, Express API.

5. Ranked USP analysis

These are project-specific strengths and positioning hypotheses, not a competitive market study. No claim that another product lacks these capabilities is made.

USP 1 — Capacity-aware admission at the point of commitment

The strongest concrete differentiator is the transactional connection between record validation and approved intake. A record can be structurally valid and still be inadmissible because the relevant pool is full. EduVerify implements that distinction in the write path. Demonstrate it with two otherwise-valid rows competing for one remaining seat, followed by a query proving no partial over-capacity admission.

USP 2 — Explainable matching without exposing full identifiers in cases

The combination of keyed linkage, DOB blocking, name normalization, masked case output and explicit reasons supports an accountable review workflow. The benefit is review quality and controlled disclosure, not certainty of fraud detection. Prove it with exact matches, benign name variations and distinct students sharing similar names, then measure reviewer outcomes.

USP 3 — Two independent gates before evidence release

Verifier authority and student consent are separate checks. A verifier's organizational permission is not treated as the student's permission. Audience-bound, short-lived signed evidence then limits reuse. Prove both deny paths, audience mismatch, expiration and consent withdrawal through the backend; reconcile lifecycle issues before a full security claim.

USP 4 — A connected governance proposition

Ingestion, review, capacity and evidence release live in one repository and domain model. This gives the project a coherent product story. Today the connection is stronger in the service design than in the portal integration. Closing adapters is a better next investment than adding another dashboard.

USP 5 — Reproducible, inspectable demonstrations

Synthetic seeding, deterministic fixtures, explicit rules and tests make the project easier to explain and evaluate. This is especially useful in a hackathon review where judges should see why a result occurred. Keep this secondary to user value; reproducibility is engineering credibility, not a standalone institutional purchase reason.

Not strong standalone USPs: using React, displaying charts, adding a chatbot, using blockchain-like wording for a hash chain, having a QR/token page, or checking identifier length. Technology choices become differentiators only when they produce a measured user outcome.

6. Claims suitable for presentations

Use this wording	Avoid this wording
Synthetic APAAR-oriented record management prototype	Official APAAR verification platform
Flags explainable duplicate candidates for human review	Detects fraud with guaranteed accuracy
Backend enforces configured intake at commit	Guarantees nationwide regulatory compliance
FastAPI signs consent-scoped WEAVE record checks	Every portal certificate is cryptographically verified
Masked outputs and keyed identity linkage	No personal data is stored
Hash-linked audit foundation	Immutable, tamper-proof blockchain
Implemented backend with incomplete portal integration	Fully production-ready end-to-end system
Advisory assistant	AI makes admissions or identity decisions

30-second pitch: Institutions need more than a place to upload student records. They need to understand errors, investigate overlaps and avoid exceeding approved intake. EduVerify brings those checks into one prototype, with masked duplicate review and consent-scoped signed record checks. Its FastAPI foundation implements these mechanisms; the current milestone is completing portal integration and proving the workflow with synthetic data.

7. Priority gaps and acceptance criteria

P0 — Before claiming a functioning live portal

1. **Preserve identity and tokens.** Remove fixture substitution from live login and student lookup. Accept only the backend principal. Pass separate institution, regulator and student sessions without fallback records or synthetic access tokens.
2. **Unify endpoint contracts.** Replace legacy conflict/attestation/audit calls or explicitly isolate them in mock mode. Run each supported screen against FastAPI without unexpected 404s, empty fallback data or hidden local writes.
3. **Unify permitted claims.** Align policy, CLI, routes, service and shared contract. Demonstrate every supported claim through grant, consent, issuance and verification. Remove or redesign award claims until backed by actual award evidence.
4. **Make lifecycle policy explicit.** Decide whether grant withdrawal invalidates existing credentials immediately; define purpose-specific consent withdrawal and consent narrowing. Test the chosen semantics against PostgreSQL.

P1 — Before a reliable institutional pilot

1. **Prove tenant and student isolation.** Attempt cross-tenant IDs, forged role fields and another student's identity reference. Expect denial without disclosing protected records.
2. **Prove commit concurrency.** Submit competing batches into a nearly full pool and verify the stored total never exceeds approved intake. Re-upload and recommit to prove idempotency.
3. **Complete durable student workflows.** Back discrepancies, profile corrections, consent preferences and request history with authorized endpoints. A saved action must survive reload and be visible to the intended second actor.
4. **Harden audit behavior.** Standardize writers, cover duplicate decisions and test concurrent first/tail writes. Add a verified read/export path with tenant policies and a trustworthy external anchor or retention strategy as needed.
5. **Rehearse deployment.** Exercise migration, seed/provisioning, login, upload, commit and signed check in the deployed topology. Health endpoints alone are insufficient.

P2 — Before scale or broader rollout

1. Implement the separately planned worker and cancellation/retry semantics; profile duplicate-block growth and upload contention.
2. Add keyboard/screen-reader and responsive checks to both portals; validate actual user comprehension of warnings and consent.
3. Define operational key custody, rotation, retention/deletion, backups, recovery, shared throttling and monitoring. Validate each operational procedure rather than relying on configuration names.

These priorities are recommendations from the review, not changes made to application behavior in this documentation update.

---

8. Demonstration plan

Preparation: use the synthetic seed manifest, two fictional institutions with explicit capacity, provisioned local accounts and a private signing key. Never use real identifiers. Keep API demonstrations separate from fixture portal demonstrations.

1. Open the deployed landing page and explain the student/admin roles. Label the portal as a synthetic experience.
2. In local fixture mode, tour the student record and discrepancy screens. State that these actions do not yet demonstrate backend persistence.
3. Through FastAPI docs, authenticate as an institution operator and upload the valid generated CSV. Inspect structured findings, then commit eligible rows.
4. Upload the same canonical content under another filename. Show that it resolves to the prior batch and does not create duplicate enrolments.

5. Upload the generated invalid and capacity scenarios. Explain the distinction between malformed input, row review and capacity failure.
6. Upload the second institution scenario and run the backend duplicate scan. As a regulator administrator, inspect masked evidence and record a human outcome.
7. Provision a verifier grant and student consent for `enrolment_status`. Request a credential, verify it for the correct audience, then demonstrate a wrong audience and withdrawn consent.
8. End with the feature matrix and integration roadmap. Do not imply the previous API steps were all driven by the current frontend.

Suggested screenshots for later additions: a real local fixture admin dashboard, a backend validation result, a masked duplicate case and a consent denial. Label environment and capture date. Do not fabricate UI screenshots or include credentials or full student identifiers.

9. Measurement plan

No performance or quality numbers are claimed by this assessment. Use these metrics to turn implementation into evidence:

Measure	How to evaluate	Why it matters
Validation repeatability	Repeat a fixed corpus with a pinned ruleset; compare row outcomes	Explains and reproduces decisions
Duplicate precision/recall	Label synthetic exact, variant and non-match pairs; report per match type	Quantifies false positives and missed overlaps
Reviewer effort	Time matched-case review and record disagreement rates	Tests whether explanations help users
Intake correctness	Concurrent commits near capacity; inspect final database counts	Validates the central write invariant
Upload idempotency	Rename/reorder columns/retry; compare batch and enrolment counts	Prevents accidental repeated writes
Authorization correctness	Role-by-resource denial matrix with cross-tenant identifiers	Tests backend trust boundaries
Credential lifecycle	Invalid signature/audience, expiry, withdrawal and revoked grant cases	Establishes actual evidence-release semantics
Workflow persistence	Save, reload, re-login and inspect as another authorized actor	Detects fixture-only behavior
Performance	Upload/scan p50 and p95 latency by row count and block distribution	Supports scale claims with workload context

Do not publish a single accuracy percentage without stating the dataset, class balance, thresholds and whether outcomes were manually adjudicated.

10. Architecture and delivery assessment

The current system has three state locations: Express memory, browser memory and PostgreSQL. That is useful for independent demonstrations, but confusing if the UI presents them as one source of truth. A deliberate boundary between fixture mode and backend mode should be the immediate architectural objective.

The root frontend uses React 19, while accepted decisions still mention React 18 and `apps/web`. The separate worker is planned but absent. Vercel service configuration exists, and public synthetic health checks respond, but deployment health is not evidence that secret configuration, migrations and every route are working. Shared OpenAPI and design documents need reconciliation with actual routes rather than being treated as an automatically current specification.

The next milestone should be narrow: **one correct institution upload/commit flow, one reviewed duplicate case and one consent-scoped record check, all driven by authenticated backend state**. This creates a stronger reviewable result than expanding surface area before integration is complete.

11. Evidence index and review limits

Area	Primary source
Active UI routes	src/App.tsx
Fixture identity and portal mutations	src/lib/api-client.ts
Mock sessions and legacy endpoints	server.ts
Simulated signatures and validation	src/lib/weaveEngine.ts
Auth and policy	auth routes, policies
CSV and persistence	parser, ingestion
Duplicate review	duplicate service
Consent and credentials	verification, privacy routes
Audit assurance	audit service, privacy tests
Deployment and environment	Vercel configuration, environment template, Compose
Architecture intent	DECISION.md, RULEBOOK.md

This is a code-grounded product and implementation assessment, not a penetration test, legal opinion, competitor survey or certification. Findings marked as inferences need reproduction. The documentation changes do not repair the identified runtime issues. Refresh this report when the integration boundaries change.